Overview

Distributed computing

- Software components on separate physical machines
  - Client-server architecture
  - WEB application
  - Computation on a cluster
  - etc.
- Communication over network
- Different application domains
  - High-performance computing
  - Enterprise applications

This course is devoted to a specific application domain in distributed computing. Distributed computing is a broad area in computer science and engineering. The main concept is that we develop software the components of which are executed on distinct physical computers. The machines are connected by some network, either LAN or WAN, and the components of the system communicate over this network, for instance using TCP/IP. During the course we shall use the Java programming language, as well as technologies related to this language.

The most typical instance of distributed computing is the client-server architecture, in which we allow (typically multiple) clients access the services of a server. Client and server components are different in complexity, and the computers that execute those components are different as well. Computers executing the server code are expected to be more powerful than the ones executing the client code.

WEB applications are extremely wide-spread today. These are client-server applications with web browsers playing the role of the clients. The complexity of the code executing in the web browser is varying. In one extreme, it can be plain HTML with forms. However, real programming languages are also supported in web browsers, most notably JavaScript, and even Java (although the technology for the latter, i.e. Java applets, are considered obsolete today). We can emphasise here another important property of distributed computing: different components might be written in different programming languages. We will, nevertheless, use only Java.

Another well-known scenario for distributed computing is when we execute components of the software on a computer cluster. There are many technologies supporting the construction of clustered applications. We shall meet one such technology in this course. Maybe it is worthwhile mentioning here that computer systems made available for mass execution of (potentially clustered) applications are also actively

utilized. These belong to the fields of grid computing and cloud computing, and we will not consider them in this course.

There are many application domains in which distributed computing plays a crucial role. One can use distributed computing for achieving high performance, such as in the case of supercomputing. Number crunching, scientific computations, compute-intensive industrial applications, simulations all fall into this category. Distributed computing is also relevant for inherently geographically distributed problems. During this course we will focus on a narrow domain, on distributed enterprise applications.

Enterprise Applications

- Distributed systems
- Implement enterprise processes
- Heterogeneous
  - Server, PC, mobile device
  - UNIX, Windows, Mac, Android, iOS ...
  - TCP/IP, HTTP(S), SOAP, IIOP ...
- Databases, information systems
- Legacy systems

Enterprise applications are distributed applications implementing enterprise processes. They are generally used in some heterogeneous hardware-software environment, where different platforms coexist. Communication is organized via different protocols. Data is stored in databases (in most cases in RDBMS, i.e. relational database management system), or some other kind of information systems. In some applications cooperation with some external systems (legacy systems, business partners) is an important issue.

Issues to deal with in enterprise applications

- *Business logic* – might be complex
- *User interface* – should be ergonomic
- Other issues?

Other issues

- Persistence

- Transaction handling
- Load-balancing
- Authentication
- Authorization
- Fail-over
- Communication
- Multi-threading
- Resource-pooling
- Caching
- Clustering

A common enterprise application must deal with some different issues. First of all, there is the application logic itself, which is often called "business logic". This might be fairly complex in a reasonably sized enterprise application. Then there is a user interface, maybe different user interfaces for different users. The UI must be designed to be ergonomic. Are there any other issues?

Well, yes! Look at the long list of other issues that a real enterprise application must deal with. We must store data in persistent storage, for instance in a RDBMS. To allow multiple simultaneous clients access the data, transactions should be provided. To make good use of available hardware resources, load-balancing is required. Users of the application must be authenticated, and care must be taken to allow them to use only those services that they are authorized for. If something goes wrong, the show must go on – preferably without crashing other activities, and retrying to serve requests in case of failures. Since our applications are distributed, communication must be facilitated among components; transparent remote access is a nice way to achieve this goal. Server components must be able to serve many simultaneous clients – this is often achieved by serving each client in a separate thread, so that they do not block each other. If resources must be shared among many clients, resource pooling can improve performance. Caching and clustering are also for this purpose – we shall be especially interested in the clustering issue during this course.

It might seem that these "other issues" require more design and programming than the business logic and the user interface. However, these other issues can be handled very similarly in all applications. It is a good design to separate these general concerns from the application specific parts. One could even develop solutions for the "other issues" in a reusable way. A company could develop its own set of software components for this purpose. These components can even be organized into a framework: in this case business logic and UI can be programmed as components running within that framework. This could extremely speed up application development.

An interesting idea is to design the solutions for the "other issues" above in a way that is reusable even more, namely that the solutions are not tied to a specific company. Can we define these as standard services that different frameworks can provide and rely on? Some stakeholders in the software industry, including framework vendors, decided to develop a standard for implementing services for frameworks, and application programming interfaces to access these services.

Java Enterprise Edition

- Specification
  - To create a framework for distributed enterprise applications
  - To write software that can execute in such frameworks
- Implementations
  - Oracle Java EE SDK (Glassfish ES)
  - WildFly (JBoss AS)
  - Oracle WebLogic
  - IBM WebSphere

This is how the enterprise technologies for Java have emerged. The Java Platform Enterprise Edition (Java EE for short) is a specification created by the Java Community Process. As of writing, the most recent version is Java EE 7 (JSR 342, 12th June, 2013). This is a de facto standard; implementations conforming to this specification are called Java EE compliant.

There are numerous compliant application servers, both proprietary and open source ones. The most important open source implementation is GlassFish, which is the reference implementation, and part of the Oracle Java EE SDK. Another one is WildFly (formerly JBoss Application Server), which is now maintained by Red Hat Inc.

All this is extremely convenient for Java developers. System architects can rely on well established solutions, and software engineers can write the code of enterprise applications focusing only (or at least, mostly) on the application specific issues. Moreover, there is no vendor lock-in; an application written against one Java EE compliant framework can be executed on any other Java EE compliant framework as well.

Of course, the specification does not cover every little detail of middleware services for enterprise applications. As time passes by, more agreements are made within the community, and each version of the specification defines more details. However, there are still open questions to agree upon, and hence some minor adjustments might be required when migrating an application from one framework to another. But these minor problems do not obstruct us too much.

Activities for enterprise application development

- Develop components
- Assemble components into applications
- Deploy applications or components
- Administer applications
- Administer servers/clusters

There are many activities with enterprise applications that the Java EE specification is concerned with. In this course we will focus on how to design and develop the components of such applications, how to assemble these components into applications, how to deploy the components separately, or a whole application on a so called application server, and how we can administer running applications and the servers themselves.

Since we are talking about components here, we should first get familiar with some relevant concepts of component-based software development.

Component-based software development

Component

- Solution for a problem
- Loosely coupled units of functionality
- Component is similar to module and object
  - Strong internal cohesion
  - Well-defined, small external interface
- Organizing objects into scenarios
- Different component models exist
  - CORBA, DCOM
  - OSGi
  - JavaBeans is not Enterprise JavaBeans

Components are loosely coupled building blocks of software systems. A component is a solution for a specific problem, and by composing these components, more complex problems can be solved. Components have a well-defined interface. Other components can access the component through this interface. Replacing a component with another one with the same interface must be possible. As a component is responsible for solving only one problem, it usually has strong internal cohesion.

There are many component models used in the industry. The most well-known is probably the CORBA Component Model from the Object Management Group. The Microsoft distributed component model, DCOM, can be parallelled. For the Java programmer, OSGi should be stressed out. Finally, we hear a lot about JavaBeans and Enterprise JavaBeans. They should not be confused: Enterprise JavaBeans is the server-side component model for distributed enterprise applications, and completely different from JavaBeans (which is typically considered as, but not exclusively used in, client-side). This course will put much emphasis on describing Enterprise JavaBeans.

Component-oriented programming

Programming = Configuring and assembling components

- Build from existing components
- Communication/interface is standard
- Components can be replaced
- Reusability

Components can be:

- Configured
- Specialized
- Customized

In component oriented programming, we assume that many components are already available off-the-shelf. Hence, programming is mainly concerned with integration, which includes the configuration, specialization, customization of components, as well as their assembly into a software system.

It is favorable to use standard ways for organizing communication among components. Their interfaces should be defined with that in mind. This will also facilitate the replacement of a component with another conforming one. The possibility to easily replace a component with another one is a central issue in component-based software development.

Examples for components

WebShop Application

- Displaying of goods
- Customer database
- Shopping cart
- Payment processing

Payment processing component

- Parametrized by the bank for credit card processing
- Knows how to communicate with that bank

To create a web-shop application, we could rely on some existing components: a component to display items, another component implementing shopping cart functionality, a component to hold the customer data, and a component to process credit card payments.

In this example, the payment processing component might require parametrisation by the bank which handles the credit cards. After parametrization, the component is responsible for staying in communication with the bank.

Multi-tier applications

3-tier applications

- Presentation tier
- Logic tier
- Data tier

In what follows, we shall see that Java enterprise applications are structured into so-called tiers. This is a general concept, often found in the architecture of complex software systems. Client-server applications are frequently organized into three tiers: the middle tier implements the main logic of the application, and relies on the data tier. User interactions are handled by the presentation tier. Let's investigate what the concept of tiers covers.

Multi-tier application

- Layered architecture
- Physical separation of layers
- Separate computers/clusters
- Central concept for Java EE

Complex software systems often have a layered design. A layer is a logical structuring of the system. It is typically orthogonal to the provided functionalities/features (another concern for structuring), as well as to the processes that make up the dynamic structure of the system. A layer modularizes a set of related activities that the implementation must perform. A typical layer is the data persistence layer: all operations which manipulate directly the database used by the application can be grouped into this layer. Many features provided by the application may need persistent data: the implementation of those features will contain operations from the data persistence layer. Many processes that are executed in the system may access the database: again, the implementation of those processes will therefore contain operations from the data persistence layer.

The term *multi-tier* (sometimes also referred to as *n-tier*) means that the layers of the system might be physically separated. Each layer is implemented as components. Often they can also be deployed on different physical machines. It may happen that a layer is deployed on a dedicated computer cluster. Since the activities carried out in different tiers have different hardware (and OS) requirements, it is arguably a good idea to deploy the different tiers on different machines, deploying each tier on a platform that is best suited for that tier.

This multi-tier design is a focal concept in the Java Enterprise platform.

3-tier enterprise applications

- Presentation tier
- Business tier
- Enterprise Information Systems tier

In the Java EE context, the general 3-tier model is characterized with the separation of presentation logic, business logic and information systems. The terminology used here adapts to the domain, as we can see. Furthermore, instead of data tier, we say EIS tier, which better emphasizes that enterprise applications might rely also on information systems other than databases.

Advantages of the middle tier

- Thin client
  - Weaker hw is enough
  - Less sw to install
- Different clients
- Server-side services
- Security
- Easier performance tuning
- Reliability
- Easier maintenance
- Easier future development

The separation of the business tier offers many advantages. First, it makes thin clients feasible. Thin clients can operate on platforms with limited hardware resources (such as mobile phones), and even more importantly, without the need to install (and administer) software dependencies, and without taking too much care on operating system versions, for instance. Often a web browser is sufficient on the client side to access the functionalities of the application.

Furthermore, many different clients can be developed for the same application, all focusing on a specific use case, without duplicating code for application logic. Different clients can have different requirements: some clients prefer more sophisticated user interfaces, but agree to use more powerful client computers (rich clients). Other clients are satisfied with less sophisticated user interfaces, so that they need not allocate too much computer resources. All these different clients can be served with the same business tier.

The fact that the business logic is executed on the server side, introduces further benefits. The business logic can easily rely on services (transactions, message queues etc.) that need need to be deployed and maintained centrally, on the server. From security point of view, much of the application logic is running in a protected environment, behind a firewall. Performance tuning is achieved by adjusting the size of the server cluster. System administration becomes easier by centralization of services, which makes the operation of those services more reliable, and easier to maintain. Finally, adaptation to novel needs becomes easier as well. The modification of some functionality can be carried out at a single place, without modifying and redistributing client components. New emerging user needs can be supported by the fast development of novel thin clients.

4-tier applications

- Client tier
- Web-tier
- Business tier
- EIS tier

If we take the wide-spread use of web solutions into account, we might want to refine the 3-tier model, and introduce a fourth tier: the web-tier. We can observe this 4-tier architecture in many Java EE applications. Most of the time the role of the web-tier is to implement the presentation logic, so the client tier can be really thin: consisting of some dynamic HTML content, maybe enriched with JavaScript, executing in a web browser.

If the application is not very complex, the web-tier and the business tier can be merged into a single tier, but this is not a very scalable solution. If we plan long life-time for the application (and plan for scalability as the number of users is expected to grow), we separate the web tier from the business tier.

Creating Java EE applications

- Components can be purchased or developed
- Customizing and assembling components
- Middleware is standardized and ready to use

Benefits

- Fast and cheap development
- Robust components
- No vendor lock-in
- Maintainability, modifiability, scalability

In a Java EE application, development can be done by purchasing, customizing and assembling components, this way minimizing the time of, and effort needed for, development. To tell the truth, the use of Commercial-Off-The-Shelf Software (COTS) is not a viable option in many cases: Java EE developers often need to develop (and, in the positive case, can reuse) the necessary components.

In any case, the availability of middleware for supporting the many important aspects of large-scale, complex, reliable, secure and scalable software systems proves to be a valuable help for enterprise software developers. They can focus on implementing the business logic and the presentation logic of the application, and simply rely on the existing middleware services. The wide range of facilities provided by Java EE compliant frameworks makes application development faster and cheaper. Components will be easier to reuse or modify, applications will be easier to maintain, and they can easily be made scalable.

Finally, note that the services offered by the middleware are standardized: multiple implementations are available, and these implementations are interchangeable. This way vendor lock-in is not a threat anymore.

Anatomy of a Java EE application

Components

- Client
- Server
- Web

Containers

- Hosting components
- Middleware services

Java EE applications consist of components. According to the multi-tier approach, the components can be categorised as client, server and web components. Each type of component has its own dedicated container type. For instance, a web component runs in a web container, where the container is responsible for the life-cycle of the component, supports the incoming and outgoing communication of the component, and provides the component certain middleware services. The middleware services provided by a container is specific to the type of the container.

Anatomy of a Java EE application

The bottommost tier, the Enterprise Information Systems tier can consists databases, for instance, and deployed on one or more database servers. The EIS tier is not particularly part of the Java EE standard, but connection to an EIS provider is an important service of containers.

The business tier is typically implemented as Enterprise JavaBeans, running in EJB containers. The web-tier can be implemented with servlets, Java Server Pages and

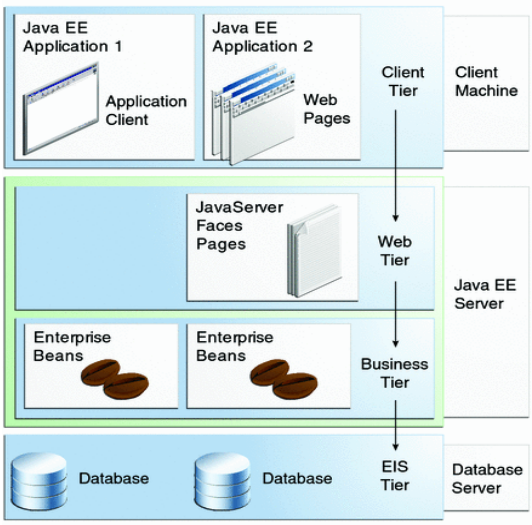


Figure 1: Tiers in a Java EE Application
Image is from the Java EE Tutorial

Java Server Faces. These are executed in web containers. The business tier and the web tier reside in the so called Java EE application server. The application server can be distributed in a cluster of server machines.

Finally, in the client tier we can have thin and rich clients. Thin clients can be web pages (possibly with some JavaScript code) running in web browsers. Rich clients can be, for instance, application client components running in application client containers. Clients are executed, obviously, on client machines.

Client tier

Web clients

- Web browser (HTML, XML, WML)
- Thin client

Applets

- Java Plug-in
- Security policy

Rich clients

- GUI or command-line
- JNLP

There is a wide variety of clients in Java EE. Some clients can be defined as components and run in a container, so that they can enjoy the services offered by the container: these are application client components. However, clients can be written as standalone client programs as well. Clients of these two kinds are typically rich clients, with graphical user interface. However, they can also be less sophisticated, providing only a command-line user interface. The JavaBeans component model can also appear in rich clients.

In rich clients the full Java SE can be used. Rich clients must be deployed on the client machine, and the client machine should have a Java Virtual Machine installed. The deployment and the upgrade of the application can be managed centrally using JNLP.

Thin clients are typically executed in a web browser. There is another option for web browsers, namely applets. Applets are executed in applet containers. This technology has become less fashionable recently, we only mention it here for completeness. Applets run with certain restrictions, specified with the security policy of the browser. This way, software from different sources can be run safely. Using digital signatures, software can be signed, and software from trusted vendors can gain elevated access to the computer, compared to software from untrusted sources.

Client-server communication

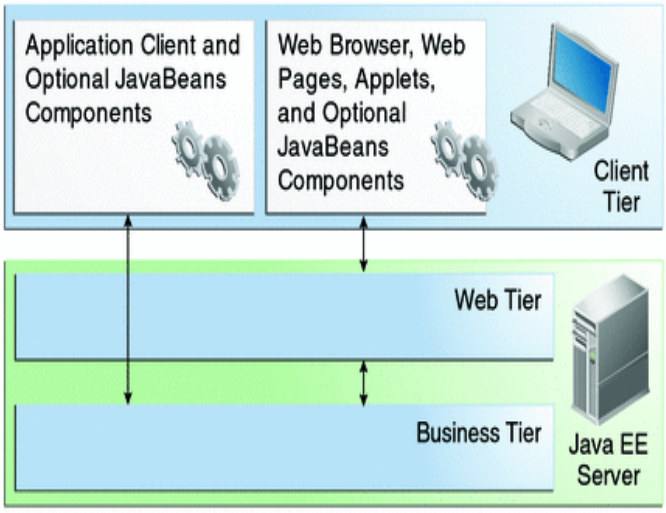


Figure 2: Client-server communication
Image is from the Java EE Tutorial

A typical thin client is a web browser, which presents documents created by the web tier. Rich clients, on the other hand, access enterprise services directly from the business tier, using remote procedure call.

Web tier

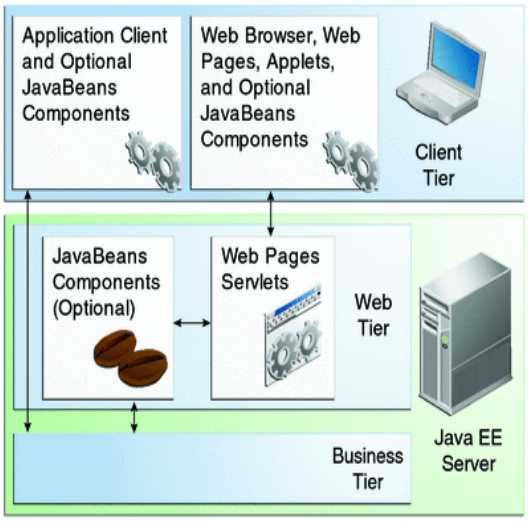


Figure 3: Web tier
Image is from the Java EE Tutorial

There are many technologies for the web tier. This course will be unfair with these technologies, because we shall put more emphasis on the business tier. The traditional method to write a Java web component is to use Java Server Pages (or lately, Java Server Faces) together with servlets. These technologies utilise JavaBeans to decouple the business tier from the client tiers.

Business tier

In the business tier we shall focus on Enterprise JavaBeans. There are different kinds of EJB components, such as session beans, message-driven beans, singleton beans, and even entity beans. However, in recent versions of the EJB specification, entity beans are made obsolate by JPA (Java Persistence API) entities.

Containers

Containers

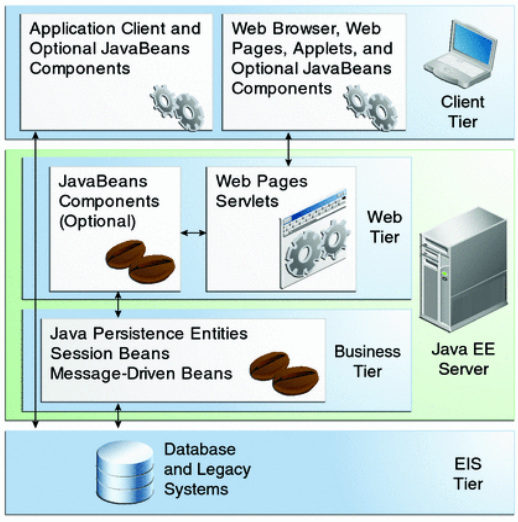


Figure 4: Business tier
Image is from the Java EE Tutorial

- Application client container
- Applet container
- Web container
- EJB container

The container concept is extremely important in Java EE. All components run in the respective container: EJB components run inside EJB containers, web components run inside web containers, and application client components run in application client containers. Even applets are executed in appropriate containers, in applet containers.

The tasks of containers

- Manage life-cycle of components based on deployment information
- Provide services for components
- Intercept communication

Much of the Java EE framework is implemented in the containers. The server side of an application is the application server, which hosts the EJB and web containers, which host the EJB and web components. When an application is deployed in the application server, its components are deployed in the respective containers. When a component

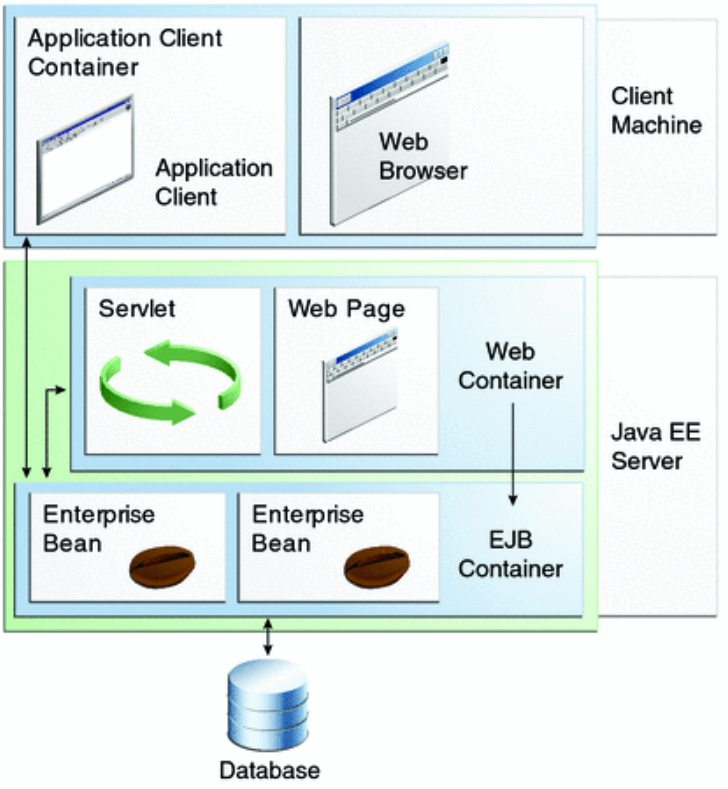


Figure 5: Server Containers
Image is from the Java EE Tutorial

is deployed, the container starts to manage its life-cycle based on the descriptions passed during deployment. While the component operates, the container provides services to it. The code of the component can rely on these services, as specified in the Java EE specification. Some of these services are accessible to the component through API calls, other services are provided implicitly. This means that the container merges these services with the operation of the component automatically. How can this be achieved? The component does not communicate with the outside world (e.g. with other components) directly. Communication is carried out through the container. This way the container can intercept all communication activities as well as life-cycle events, and insert the invocation of the required services.

Container services

- Security (authentication, authorization)
- Transaction

- Name and directory service
- Remoting, communication

Invisible services

- Life cycle management
- Persistence
- Fail-over
- Caching
- Resource pooling
- Clustering
- Multi-threading
- Load balancing

What are the services that are typically provided by containers? Services related to security, namely authentication and authorization can be explicitly accessed by the component. Transactions for manipulating different transactional resources, for example RDBMS are also provided. A widely used service is the naming service, which allows a component to locate another component to communicate with. Finally, communication through the container, and especially remote communication can also be considered as a service.

Other services are kind of invisible – the component does not handle these directly, but nonetheless, the container will take care of these. Many of the "other issues" mentioned earlier today fall into this category.

Java EE technologies

Java EE technologies

- Persistence: JDBC and JPA
- Remote communication: RMI, CORBA
- Naming service: JNDI
- Message-oriented middleware: JMS
- Enterprise JavaBeans component architecture

- Transaction management: JTA, JTS
- Web programming: Servlets, JSP and JSF
- Web services: JAX-WS and JAX-RS
- etc.

There are many technologies that Java EE application developers use. The specification standardizes the application programming interface (API) to these technologies. This course will present some of these technologies and APIs, but not all of them. In this introductory lecture we shall briefly mention some of them so that the "big picture" can be formed.

JDBC

- Platform independent database access
- Using SQL in Java programs
- Not embedded SQL
- JDBC drivers

The Java DataBase Connectivity (JDBC) API provides access to relational databases in a fairly platform-independent way. JDBC drivers allow us to decouple Java code from the details of the database implementation. The Java code uses the JDBC API to pass SQL commands to the database. Since these SQL commands can be (and often *are*) constructed dynamically as String values, the approach is quite error-prone.

Object-relational mapping

- Simplified database manipulation
- Database management closer to OOP
- Classes as tables, objects as rows
- Object-relational impedance mismatch
- Java Persistence API, JPA
- Popular implementations (Hibernate, TopLink, OpenJPA, EclipseLink)

The Java Persistence API provides a higher-level facility for managing relational data. It is typically used atop of JDBC. JPA supports the object-relational mapping approach, which means that we use relational data as objects. Tables correspond to classes, data rows in tables correspond to objects of the respective class. Foreign keys in tables are mapped onto references in objects. This mapping between the object-oriented world and the relational world is nice and simple in nice and simple cases, but turns out to be very hard to give in more complex cases.

Remote Method Invocation (RMI)

- Remote object: its methods can be invoked from other virtual machines
- Similar to, but also different from local method invocation
- Java-Java communication
- Based on object serialization and custom class loaders
- JRMP: Java Remote Method Protocol
- java.rmi package

Communication is one of the key challenges in distributed systems. One way to communicate in Java is Remote Method Invocation. In this approach objects can be instantiated in different Java virtual machines, and their methods can be called from other VMs in a way that is very similar to ordinary (local) method invocations. This provides a convenient communication facility, which aligns well with object-oriented style and notation. Behind the scenes advanced mechanisms take place, including object serialization over TCP sockets, remote class loading etc. The underlying Java Remote Method Protocol, implemented in the `java.rmi` package allows Java objects to communicate with other Java objects in a distributed system.

Java and CORBA

- Communication with non-Java remote objects
- Heterogeneity
- Object Management Group, Object Management Architecture
- IDL: Interface Definition Language (Java IDL mapping)
- Common Object Request Broker Architecture

A very similar approach is to rely on the CORBA (Common Object Request Broker Architecture) standard for object distribution. This is a widely supported standard, which can be applied from many programing languages, and thus provides language heterogeneity in our application. Java objects can communicate with C++ objects, or with Ada, COBOL etc. components – in fact all languages that are given a mapping to the Interface Definition Language of CORBA can be used together in a distributed application. The standard is maintained by the Object Management Group, which is an association of many leading IT companies.

RMI over IIOP

- javax.rmi package
- RMI meets CORBA: advantages from both
- Communication: IIOP (not JRMP)
- EJB uses this

Both RMI and CORBA have advantages over the other. However, it is also possible to combine them, taking some good properties of the two approaches. Remote Method Invocation can be implemented over the communication protocol used by CORBA. This protocol is called IIOP (which stands for Internet Inter-ORB Protocol). This implementation of RMI can be used from the `javax.rmi` package. Note that Enterprise JavaBeans, which is the most important technology of Java Enterprise Edition, relies on the IIOP-based implementation of RMI for transparent communication.

Naming service

- Registering objects, accessing them by name
- JNDI: directory services as well
- Internet DNS, NIS, RMI registry, CORBA namespace, LDAP (Lightweight Directory Access Protocol), Novel DNS, file system
- JNDI connector can be written for any service
- Connectors for the most popular services already exist

Before communication, components need to "find" each other: they need to obtain a reference on each other, so that they can invoke each other's methods. A naming service allows us to store references to remote objects, and to look up those references by symbolic names. Java Naming and Directory Interface (JNDI) is a standardized application programming interface (API) for accessing different naming services. There are many such services used in distributed applications, including for instance the naming service for RMI, the one for CORBA, or directory services such as LDAP. JNDI provides a unified API to bind references to names , and retrieve references by name, with all those naming services that have JNDI connectors. Many popular services have such a connector.

Message-oriented middleware

- Loosely coupled system
- Communication by message passing
- Asynchronous, persistent, off-line
- Java Messaging Service
  - Message queue
  - Publish-subscribe
- EJB: message-driven beans

Remote method invocation (or, more generally, remote procedure call, RPC) is one style for communication in a distributed system. Another style is to use a message-oriented middleware to send and receive messages asynchronously. Due to this asynchronous nature of message passing, loosely coupled systems can be constructed with this communication paradigm. A messaging middleware may provide further services, such as persistent messages and off-line communication. Here "off-line" means that communication components may become unavailable, or may go down and later up again, but the messages can still be delivered to them (when they are up and available again). The Java Messaging Service (JMS) provides a standard API for different messaging middleware implementations. JMS allows both the point-to-point style communication (message queues) and the publish-subscribe model. In Enterprise JavaBeans, JMS is used to develop the so called "message-oriented beans".

JavaMail

- Machine-human communication
- E-mails
- Asynchronicity and off-line communication

Not only software components may want to exchange messages in an application. Sometimes machine-human messaging communication is also required. The software may want to send an email to a user. This is supported by the JavaMail API. Java Enterprise Edition provides application components an email service provider as well.

Enterprise JavaBeans component model

- Java server-side component model (cf. JavaBeans)
- Implementation of business logic
- Hiding details of infrastructure
- EJB 3 and EJB 2.1
- Session, singleton, entity and message driven beans

In this course the main focus will be on the Enterprise JavaBeans technology. This is a component model for distributed server-side, and not to be mistaken with the JavaBeans model. EJBs are typically used to implement the business logic of an application. With EJB, one can focus on the business processes, and many details of remote communication etc. are hidden by the technology. There are different versions of the EJB specification. One must know something about EJB 2.1, but our major concern will be EJB 3. In EJB 3, session, singleton and message-driven beans can be developed. Furthermore, EJB 2.1 provides entity beans as well.

Transactions

- Synchronization of components
- Transactional resource managers
  - Databases
  - Message queues
- Atomicy, Consistency, Isolation and Durability
- Commit and rollback
- Local and global transactions
- JDBC transactions, Java Transaction API (JTA)
- Distributed transaction processing (X/Open XA)

One of the main features of EJB is the rigorous use of transactions. Enterprise applications often use transactional resources, such as relational databases and message queues. These resources are made available by resource managers. Transactions allow us to access these resources with the ACID properties guaranteed. We will learn about these Atomicity, Consistency, Isolation and Durability properties later.

The main goal of transactions is to synchronize the simultaneous processes of a distributed application at the resource level. Units of work can be committed in the

resource managers when they succeed, and can be rolled back when they fail. This is simple to use, but quite complex in the implementation. Especially global transactions, ones addressing more than one resource, need sophisticated transaction managers. There are different APIs to program transactions in an enterprise application. For example, JDBC provides local transactions, and Java Transaction API (JTA) supports global ones.

Distributed transaction processing is standardized. X/Open XA is such a standard.

Java servlets

- Small applications running in a web server
- Request-response protocol (usually HTTP)
- Java EE: Servlet container
- Presentation logic

We can see servlets most often as small Java components running in a web-server. The technology is more general, applicable to different request-response systems, but most often HTTP (Hyper-Text Transfer Protocol) servlets are used. In Java Enterprise, we will run servlet components in servlet containers, and they typically implement the control process of the presentation logic.

JSP technology

- Java Server Pages
- Markup language to embed application logic (program code) into web pages
- Information insertion into web page (usually into HTML page)
- Based on servlet technology
- Separated code and design

Java Server Pages (JSP) compliment servlets in the implementation of the application's presentation logic. JSP allows us to insert instructions into web-pages in order to express application logic to construct or format dynamic content. Java Server Pages are translated into servlets in the background, and executed in a servlet container. However, the programmer can separate code and design elements of the software by putting more code-like stuff in servlets, and more content-like stuff into JSPs. Hence, JSPs can be developed by web-designers, while programmers can focus on the code written in servlets.

XML

- Extensible Markup Language
- Guidelines and rules
- Creation of Data Description Languages
- Exchange of semi-structured information
- Simple text
- Portable data exchange

XML, that is data described in the Extensible Markup Language, is surprisingly popular for exchanging information between programs, services, and even human actors (such as programmers). It is a theoretically human-readable (or at least textual) format, although in practice it is better suited for machine-machine communication. XML introduces guidelines and rules, and allows us to create domain-specific or application-specific data description languages. These languages are suitable to describe semi-structured information: it is especially well-suited for organizing data in a tree structure. Due to its textual nature, XML is considered a portable data exchange format.

In Java Enterprise Edition, XML is often used for describing configuration information for components, but it is also used in data exchange in the communication between components.

JAXP: Java API for XML Processing

- Processing of XML documents
  - Document Object Model (DOM)
  - Simple API for XML Parsing (SAX)
  - Streaming API for XML (StAX)
  - XML Stylesheet Language Transformations (XSLT)

To process XML documents, different APIs are available. Collectively, we talk about JAXP, i.e. Java API for XML Processing. This provides us different XML parsing APIs, as well as API for XML transformation with XML Stylesheet Language Transformations (XSLT). Furthermore, JAXP offers name-space support to eliminate name conflicts in schemas, and allows us to use the W3C schema.

One way to parse XML is to use the Document Object Model. In this approach the content of an XML document is loaded into memory, and represented as Java objects referencing each other in a tree-like manner. This object graph is easy to traverse by

the application. We use this DOM approach only for smaller documents, since the DOM of larger documents may not fit into memory at once. DOM uses a predefined set of Java types to represent any XML data in the object-oriented world. A slightly better, more convenient approach is to allow applications introduce their own set of classes, and bind the XML schema to these classes. In this way an XML document can be parsed into an object graph constructed from custom classes. This approach is called Java Architecture for XML Binding (JAXB).

The Simple API for XML Parsing is suitable for large documents as well. In this approach, the parser scans through the XML document, and when a given structure is recognized, it invokes a call-back routine. These call-back routines are provided by the programmer. SAX is therefore an event-based parsing facility.

Another parsing API for large documents is the Streaming API for XML (StAX), This allows us to parse (and modify) an XML documents in a pull-style. This means that, in contrast to the push-style SAX, the client code calls the parsing library, to pull data from the document. In push-parsing, the parser calls the client code, hence the thread of control is directed by the parser in push-parsing, and by the client in pull-parsing.

Web services

- JAX-WS
- JAX-RS

We will learn about two technologies for implementing web-services. The older JAX-WS (Java API for XML Web Services) relies on encoding requests and responses in XML documents. This technology allows us to bind XML data to Java objects using JAXB. JAX-WS has client API for accessing web-services, and it also has techniques for implementing web-service endpoints.

JAX-RS (Java API for ReSTful Web Services) is a more recent, and more lightweight technology for web-services. Representational State Transfer (ReST) is a client-server model in which the server is stateless, and the state of the application is transfered in requests and responses.

Security

- Authentication and authorization
- JAAS
- Users, groups and roles

One very important concern in enterprise application is security. The Java Authentication and Authorization Service, integral part of the Java Enterprise platform, gives

us techniques to authenticate users, as well as authorize either single users or user groups. Authorization can be specified in a role-based manner. Different user roles can be granted permissions to perform certain actions or access certain data. Users and user groups can be assigned different roles.

The authentication and authorization logic can be separated from the main application logic, and described in a declarative way. JAAS also provides API for programmatic user authentication and authorization.

Production roles

Roles

- Different activities - different skills
- Different skills - different people?
- Sometimes same person does all
- Responsibilities set out
- Input, activity, output

The development of an enterprise application is a complex task that requires many different skills. Sometimes it is beneficial to let different activities be performed by different people, so that each activity is performed by someone who possesses the appropriate skills. This is especially feasible at a big company. Of course, when only a small development team is available, it may happen that each person takes part in all activities. Nevertheless, the Java EE specification attempts to split up the development process, and introduces the concept of development role. Each development role is responsible for a certain activity in the development process. In order to deal with responsibilities in an accountable way, the specification describes the expected input and output of the activity.

List of roles

- Java EE Product Provider
- Tool Provider
- Application Component Provider
- Application Assembler
- Application Deployer

- Application Administrator

What are the development roles? Product providers are not strictly part of the development of an enterprise application, but rather part of the production of the framework, the production of the implementation of the Java EE specification. Similarly, tool providers make software development tools (modelling tools, IDEs etc.) available for application developers.

From our perspective the important roles are Application Component Provider, for instance those who develop EJB components; Application Assembler, who configures and assembles components into applications; Application Deployer, who further configures components and applications, and then deploy those into the application server; and finally, Application Administrator, who monitors the operation of applications and services, and takes actions when something goes wrong.

Packaging components

- Byte code, HTML, JSP, configuration files etc.
- Packaged into JAR file
  - EJB jar
  - Web archive (.war)
  - Enterprise archive (.ear)

The output from one activity can be the input for another one. To facilitate information exchange, the result of an activity is often encapsulated into a single Java Archive file. Responsibilities of development roles are set out based on these JAR files, which typically contain byte code, HTML and JSP code, configuration files (deployment descriptors) as well as further contents and resources.

When an EJB Component Provider develops an EJB component, the component will be placed in an EJB jar file, which is a JAR file with an expected inner layout. Within this JAR file one can find the byte code and the deployment descriptor.

A web component is produced as a Web archive, which is a JAR file with a special extension: *.war*. This kind of archive also has a special inner layout. Apart form the byte code of servlets, it contains JSP and HTML files as well.

The component archives can be assembled into enterprise archives (again JAR files with special, *.ear* extension) by the Application Assembler. This enterprise archive can be passed on to the Application Deployer, who is responsible for final configuration and deployment of the application.